

Group 40 Write Up Flight Tracker

Group Members:

- Callum Hughes 25382533
- Oliver Crosthwaite 25382362
- Vlad Manea 25382983
- Connor Nel 25361036
- Ryan Nichols 25340769
- Sean Moriarty 25383808

Abstract

Flight Tracker is an interactive data visualisation application we built using Processing (Java) for exploring US domestic commercial flight data. The application offers six visualisations, including delay by airline, busiest departure airport, a destination map, time of day distribution and flights by day of the week. All of these are navigable through a sidebar. A live search feature allows users to filter every chart simultaneously by airline code, airport code or even date range which rebuilds the views against the queried subset. The project demonstrates how raw data can be transformed into an accessible and useful analytical tool.

Introduction

The goal of this project was to design and build an interactive flight data dashboard using the Processing environment. Rather than simply displaying static charts our team aimed to create a navigable filterable application that would allow a user to explore flight patterns from multiple angles within a single interface.

Development was split across six components with each team member owning a distinct part of the codebase. This allowed us to have huge development and meant each chart class could be built and tested independently before integration.

Literature review

The project was built around the structure and guidelines provided by the module coordinator. We used the powerpoint's approach for the multi-screen architecture, CSV parsing and chart types. Shneiderman's information visualisation mantra "Overview first, zoom and filter, then details on demand" (Shneiderman, 1996) guided the query and filtering design. The Processing reference was consulted throughout for core functions in particular `loadStrings()` and `split()` for CSV parsing, `lerp()` for smooth animations on the homepage and `arc()` for the radial clock chart.

References:

- Murala, S. *CSU11013 Programming Project*. School of Computer Science and Statistics, Trinity College Dublin, 2026.
- Shneiderman, B. "The eyes have it: A task by data type taxonomy for information visualizations." 1996
- Processing Foundation. *Processing Reference*. <https://processing.org/reference>
- Happy Coding. *Processing Tutorials*. <https://happycoding.io/tutorials/processing/>

Project Details

The application is structured around a `currentScreen` variable that controls which chart's `draw()` method is called from the main loop. Each chart is implemented as a separate class, keeping each visualisation self-managed. A shared `drawButton()` function in the main file is reused across all screens for navigation.

Data Model

The application loads the csv on startup. Each row in the file is parsed into a `DataPoint` object which stores different fields, including flight date, airline code, flight number, origin airport, destination airport, scheduled and actual departure times, cancellation status, and flight distance. The class also handles delay calculation and also compares scheduled against actual departure time to determine whether a flight is delayed early or late. After any query is run, `rebuildCharts()` reconstructs every chart object using the filtered dataset so that all visualisations update simultaneously, pressing 0 at any time resets the view back.

Search and filter System

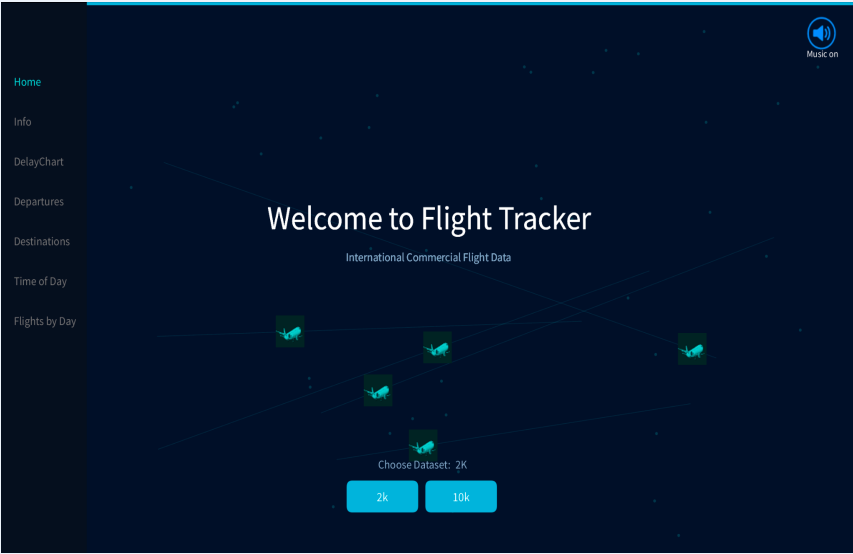
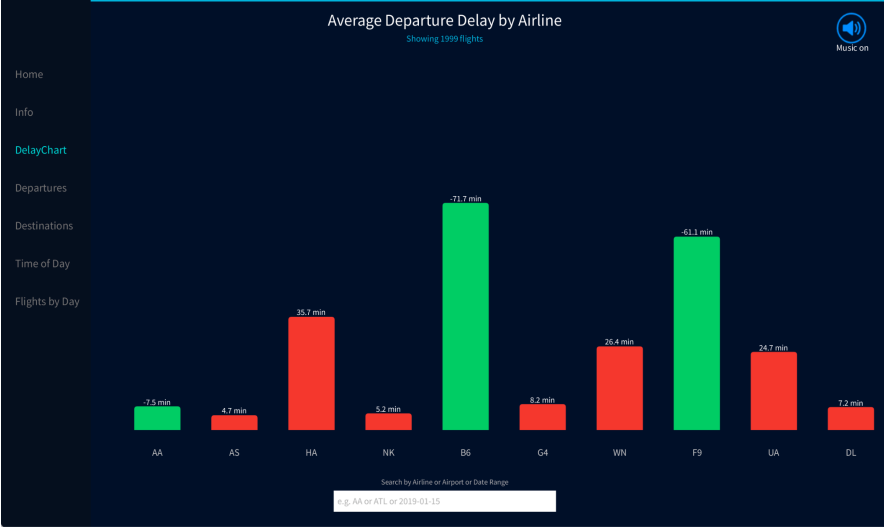
A `SearchBox` is embedded at the bottom of the sidebar, when used the `runSearch()` function in the main dispatches the input string to the appropriate query function in `Queries.pde` using the following logic:

Input Format	Query Type	Example
2 characters	Filter by airline code	AA (American Airlines)
3 characters	Filter by airport code	JFK
Contains /	Filter by date range	01/01/2022-01/31/2022

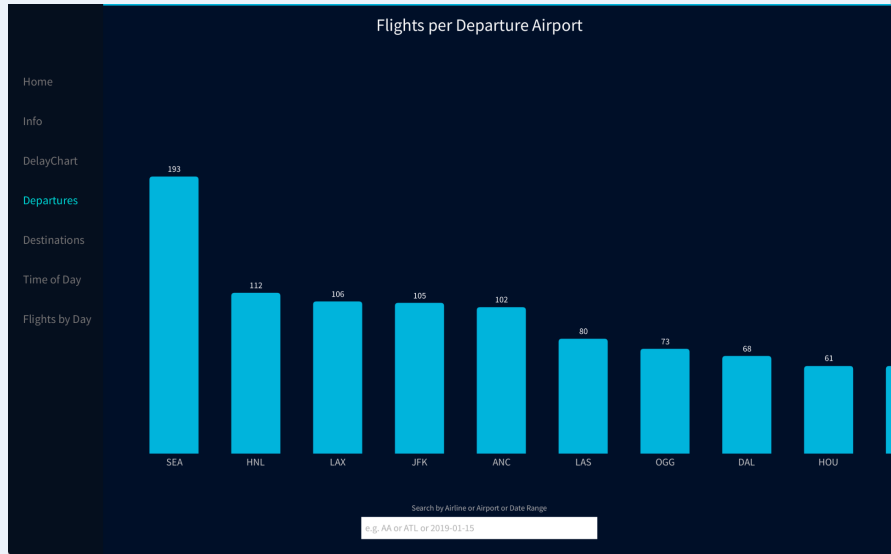
Execution and Results

Visualisations:

The application includes six screens, each offering a distinct view of the flight data:

Screen	Chart Type	Description
Home 	Animated scene	Five plane icons traverse randomised paths over a dark background, providing a visual theme.
Delay Chart 	Bar chart	Average departure delay per airline. Green bars depart early; red bars indicate those that run late.

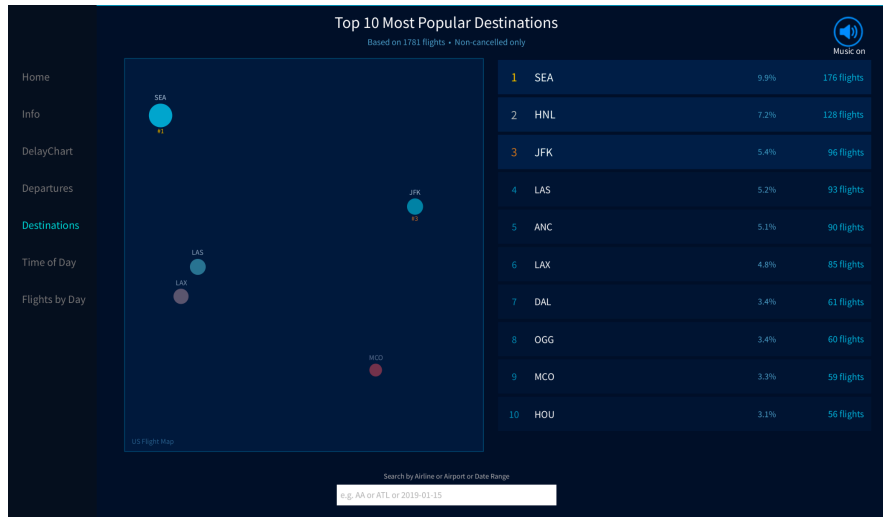
Departures



Bar chart

Top 10 busiest departure airports by total flight count, sorted in descending order.

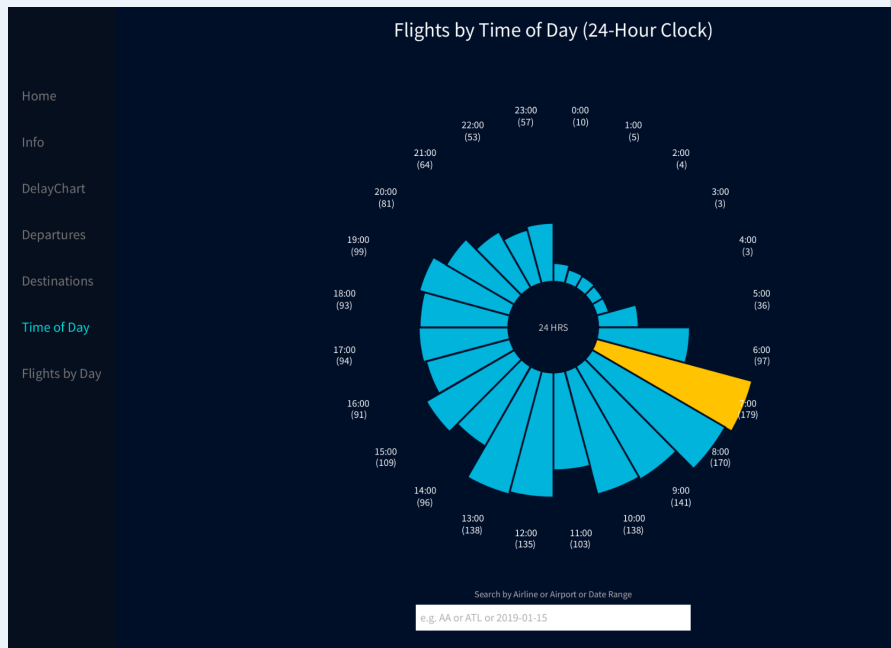
Destinations



US map + ranked list

Top 10 and also a point scatter to identify and highlight the most popular

Time of Day



Radial/pie chart

Flight volume distributed across each hour of the day on a 24-hour clock.
Busiest in gold

Flights by Date



Bar chart

Flight volume broken down by day of the week, showing which days see the most departures.

Execution:

Home Screen: The home screen connects all data pages using if/else statements to control currentScreen. Five animated planes use lerp() for smooth left-to-right motion. The data loading system reads the CSV file line by line, storing each entry as a DataPoint object. A custom parseCSVLine() method handles quoted fields containing commas to prevent them being mistaken for cell dividers, then trims and returns each field as a string array.

```

void draw() {
    if (currentScreen == 0) {
        drawHomeScreen();
        drawButtonHomePage();
    } else if (currentScreen == 1) {
        drawInfo();
    } else if (currentScreen == 2) {
        delayChart.draw();
        drawSearchBar();
    } else if (currentScreen == 3) {
        departingFlightsChart.draw();
        drawSearchBar();
    } else if (currentScreen == 4) {
        popularDestinations.draw();
        drawSearchBar();
    } else if (currentScreen == 5) {
        timeOfDayChart.draw();
        drawSearchBar();
    } else if (currentScreen == 6) {
        flightsByDate.draw();
        drawSearchBar();
    }
    drawSideBar();
}

```

```

void loadData(String filename) {
    String[] lines = loadStrings(filename);
    for (int i = 1; i < lines.length; i++) {
        String[] cols = parseCSVLine(lines[i]);
        if (cols.length < 17) continue;
        DataPoint dp = new DataPoint(cols);
        flights.add(dp);
    }
}

String[] parseCSVLine(String line) {
    ArrayList<String> fields = new ArrayList<String>();
    boolean inQuotes = false;
    StringBuilder current = new StringBuilder();
    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (c == '"') {
            inQuotes = !inQuotes;
        } else if (c == ',' && !inQuotes) {
            fields.add(current.toString().trim());
            current = new StringBuilder();
        } else {
            current.append(c);
        }
    }
    fields.add(current.toString().trim());
    String[] result = new String[fields.size()];
    return fields.toArray(result);
}

```

```

DataPoint(String[] cols) {
    flightDate = cols[0];
    airline = cols[1];
    flightNumber = int(cols[2]);
    origin = cols[3];
    dest = cols[7];
    scheduledDepTime = int(cols[11]);
    depTime = float(cols[12]);
    cancelled = int(cols[15]);
    distance = float(cols[17]);
}

```

Delay: The delay chart displays average departure delays grouped by airline using parallel ArrayLists for names, totals, and counts. After looping through all data points, each average is calculated and the bars are scaled relative to the highest value. The indexOf() method is used to check if it's a new airline to be added to the list.

```

for (DataPoint dp : flights) {
    if (dp.cancelled == 1 || dp.depTime < 0) continue;
    int i = airlines.indexOf(dp.airline);
    if (i == -1) {
        airlines.add(dp.airline);
        totals.add(dp.getDelay());
        counts.add(1);
    } else {
        totals.set(i, totals.get(i) + dp.getDelay());
        counts.set(i, counts.get(i) + 1);
    }
}

int numOfAirlines = airlines.size();
if (numOfAirlines == 0) return;

```

Departing flights: The departures chart loops through every flight using a nested loop to track airport counts. If the airport has been seen before, the count increments, and if not then a new entry is added. It then uses bubble sort to display the 10 busiest airports in descending order, then scaled them to the busiest airport.

```

class DepartingFlights {
    void draw() {
        background(5, 15, 40);
        noStroke();
        fill(0, 180, 220);
        rect(0, 0, width, 4);

        fill(255);
        textAlign(CENTER, TOP);
        textSize(22);
        text("Flights per Departure Airport", width / 2, 20);

        String[] tempAirports = new String[flights.size()];
        int[] tempCounts = new int[flights.size()];
        int numAirports = 0;

        for (int i = 0; i < flights.size(); i++) {
            String orig = flights.get(i).origin;
            boolean found = false;
            for (int j = 0; j < numAirports; j++) {
                if (tempAirports[j].equals(orig)) {
                    tempCounts[j]++;
                    found = true;
                    break;
                }
            }
            if (!found) {
                tempAirports[numAirports] = orig;
                tempCounts[numAirports] = 1;
                numAirports++;
            }
        }
    }
}

```

Day: The flights by date chart calculates the day of the week for each flight, taking into account that 01/01/2022 was a Saturday. Departures are counted into seven variables, one per day, and displayed as a bar chart

```
for (DataPoint dp : flights) {
    if (dp.flightDate != null) {
        int index = dp.getFlightDayOfWeek();
        if (index == 1) {
            satCount++;
        } else if (index == 2) {
            sunCount++;
        } else if (index == 3) {
            monCount++;
        } else if (index == 4) {
            tueCount++;
        } else if (index == 5) {
            wedCount++;
        } else if (index == 6) {
            thursCount++;
        } else if (index == 7) {
            friCount++;
        }
    }
}
```

Time of day: The time of day chart uses a radial clock design. Each hour's departure count is mapped to a slice using `arc()`, with `TWO_PI / 24` as the angle step and `map()` to scale each slice relative to the busiest hour. `HALF_PI` is subtracted from the start angle to ensure 00:00 at the top. The Processing reference was consulted for the arc and pie chart implementation.

```
class FlightsByTimeOfDay {
    void draw() {
        // Clock Math
        float clockX = width / 2;
        float clockY = height / 2;
        float angleStep = TWO_PI / 24; // one slice each hour

        // Draw the 24 pie slices
        for (int i = 0; i < 24; i++) {
            // Map the flight count to the size of the pie slice
            float sliceSize = map(hourlyCounts[i], 0, maxCount, 120, 450);

            // Calculate where this slice starts and stops
            float startAngle = (i * angleStep) - HALF_PI;
            float stopAngle = startAngle + angleStep;

            // Color the busiest hour gold, rest are blue
            if (hourlyCounts[i] == maxCount && maxCount > 0) {
                fill(255, 200, 0);
            } else {
                fill(0, 180, 220);
            }

            stroke(5, 15, 40); // Dark border around slice
            strokeWeight(2);

            // Draw the pie slice
            arc(clockX, clockY, sliceSize, sliceSize, startAngle, stopAngle, PIE);
        }
    }
}
```

Popular Destinations: The destinations page takes a top 10 list then returns a top 10 list which then the top destinations are scattered on a US map. We used insertion to organise the arrays, as this meant it could compare and insert entries to the correct position and be accurate.

```
for (String code : countMap.keySet()) {
    int cnt = countMap.get(code);
    for (int i = 0; i < 10; i++) {
        if (cnt > topCounts[i]) {
            for (int j = 9; j > i; j--) {
                topCodes[j] = topCodes[j+1];
                topCounts[j] = topCounts[j+1];
            }
            topCodes[i] = code;
            topCounts[i] = cnt;
            break;
        }
    }
}
```

Conclusion

Our project demonstrates how a raw .csv dataset can be made analytically useful and visually engaging through an interactive application. We split development by component, data modelling, file loading, rendering, individual charts and the home screen. Allowing the group to work in unison and to keep the codebase modular.

We went above and beyond in several ways. The search system accepts airline codes, airport codes and date ranges, rebuilding all six charts simultaneously from a single input. The radial clock chart and US destination map offer more creative visualisation types alongside the standard bar charts. The animated home screen, background music and colour-coded delay bars further enhance the overall user experience. Users can also select which dataset to load on startup, allowing them to choose between different data sizes to suit their needs. This gives the application genuine value beyond simple static visualisation.

Problems we encountered

Using Github when we (push) / (pull) we were encountering issues where some of ours were saying everything was up to date and some of it was not up to date. Meaning we had to restart to origin and store our code in a word document at one point.

```
PS C:\Users\Oliver Crosthwaite\OneDrive - Trinity College Dublin\ProgrammingProject> git fetch origin  
PS C:\Users\Oliver Crosthwaite\OneDrive - Trinity College Dublin\ProgrammingProject> git reset --hard origin/main
```

Future improvements

Some future improvements that we could add include increasing the data sets to 100k. As well as switching to live data, to the flights going on every day, that way our project would be able to be used in a real world environment. Expanding the project internationally, rather than just the US, would also increase the applicability of our project.